Cidades ESG Inteligentes

Documentação técnica - Automação DevOps (CI/CD, Docker e Orquestração)

Integrantes	RM
(nome do integrante 1)	(RM)
(nome do integrante 2)	(RM)
(nome do integrante 3)	(RM)

Stack: Java 17, Spring Boot 3, PostgreSQL, Docker, Docker Compose, GitHub Actions.

1. Descrição do pipeline

Ferramenta: GitHub Actions. O pipeline é disparado a cada *push* ou *pull request* na branch **main** e automatiza todo o ciclo: build, testes, empacotamento em imagem Docker e deploy em dois ambientes.

1. Build + Testes mvn verify	**2. Imagem Docker** build + push GHCR	**3. Deploy Staging** porta 8081	**4. Deploy Produção** porta 8080

Cada etapa só executa se a anterior for concluída com sucesso.

Etapa	Lógica	Gatilho
build-test	Configura JDK 17 (com cache Maven) e executa **mvn -B verify**: compila e roda os testes JUnit/MockMvc (banco H2). Publica o relatório do Surefire como artefato.	Push e PR
docker	Builda a imagem usando o Dockerfile multi-stage; em push na main, publica no GHCR com tags *sha* e *latest*.	Push e PR (push da imagem só na main)
deploy-staging	Workflow reutilizável: copia o compose via SSH, gera o .env (secrets do environment), faz *docker compose pull* e *up -d*; executa smoke test em /actuator/health.	Push na main
deploy-production	Mesmo workflow reutilizável com environment *production* (aprovação manual configurável) e porta 8080; só roda após staging OK.	Push na main

Segredos por environment: SSH_HOST, SSH_USER, SSH_KEY, DB_PASSWORD. Os ambientes são isolados por projeto Compose (esg-staging / esg-production), pasta, volumes e porta.

2. Docker

Arquitetura

Componente	Detalhe
app	Imagem construída do Dockerfile (Spring Boot, JRE 17 Alpine, usuário não-root, HEALTHCHECK). Porta 8080 no container, publicada em APP_PORT.
db	postgres:16-alpine com healthcheck (pg_isready); sem porta exposta ao host.
Rede	esg-net (bridge): app acessa o banco pelo nome **db**.
Volumes	pgdata (dados do PostgreSQL) e app-logs (logs da aplicação).
Variáveis	POSTGRES_*, APP_ENV, APP_PORT, APP_IMAGE via .env (modelo: .env.example).
Ordem de subida	app depende de db com condição *service_healthy*.

Imagem criada: build multi-stage

Estágio 1 (maven:3.9-eclipse-temurin-17) compila o projeto; estágio 2 (eclipse-temurin:17-jre-alpine) leva apenas o .jar, reduzindo tamanho e superfície de ataque.

Comandos utilizados

```
cp .env.example .env
docker compose up -d --build        # build da imagem + subida dos serviços
docker compose ps                   # status e health
docker compose logs -f app          # logs da aplicação
curl http://localhost:8080/actuator/health
curl http://localhost:8080/api/info
docker compose down                 # encerrar (-v remove volumes)
docker build -t cidades-esg:local . # build isolado da imagem
```

3. Prints do pipeline rodando

Pipeline completo (aba Actions)

[Inserir print aqui]
build-test, docker, deploy-staging e deploy-production com sucesso

Etapa de build e testes

[Inserir print aqui]
log do mvn verify com 'Tests run: 6, Failures: 0'

Etapa de deploy

[Inserir print aqui]
log do deploy e do smoke test (health UP)

4. Prints dos ambientes

Staging funcionando

[Inserir print aqui]
navegador/curl em http://:8081/api/info mostrando "ambiente":"staging"

Produção funcionando

[Inserir print aqui]
navegador/curl em http://:8080/api/info mostrando "ambiente":"production"

Containers em execução

[Inserir print aqui]
saída de 'docker ps' no servidor

5. Desafios encontrados e como foram resolvidos

Desafio	Solução adotada
A aplicação iniciava antes do PostgreSQL ficar pronto e falhava na conexão.	Healthcheck com pg_isready no serviço db e depends_on com condition: service_healthy.
Testes dependentes de um PostgreSQL real tornavam o CI lento e frágil.	Perfil de teste com H2 em memória (modo PostgreSQL), executado no mesmo job do build.
Segredos (senha do banco, chave SSH) não podiam ficar no repositório.	GitHub Environments com secrets por ambiente; .env gerado no servidor durante o deploy e ignorado pelo Git.
Nome da imagem no GHCR exige letras minúsculas.	Etapa que converte github.repository para minúsculas antes do build/push.
Staging e produção no mesmo servidor poderiam conflitar.	Projetos Compose distintos (-p), pastas, volumes e portas separados (8081 e 8080).
Container rodando como root e imagem grande.	Build multi-stage, base JRE Alpine e usuário não-root.

Ajuste esta tabela para refletir os problemas realmente enfrentados pelo grupo durante a execução.

6. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	[x]
Dockerfile funcional	[x]
docker-compose.yml ou arquivos Kubernetes	[x]
Pipeline com etapas de build, teste e deploy	[x]
README.md com instruções e prints	[ ] (inserir prints)
Documentação técnica com evidências (PDF ou PPT)	[ ] (inserir prints)
Deploy realizado nos ambientes staging e produção	[ ] (após rodar o pipeline)